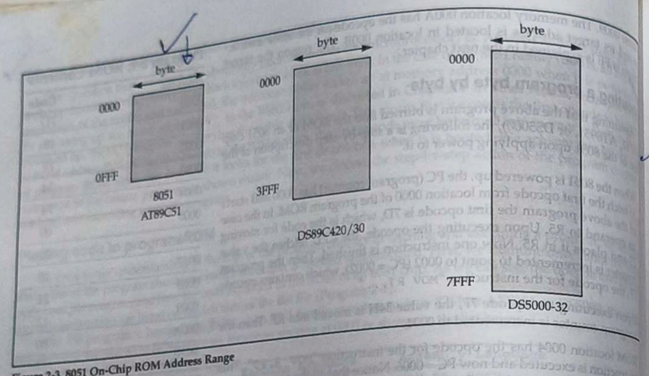



Figure 2-3. 8051 On-Chip ROM Address Range

### Review Questions

1. In the 8051, the program counter is _____ bits wide.
2. True or false. Every member of the 8051 family, regardless of the maker, wakes up at memory 0000H when it is powered up.
3. At what ROM location do we store the first opcode of an 8051 program?
4. The instruction "MOV A,#44H" is a _____-byte instruction.
5. What is the ROM address space for the 8052 chip?

## SECTION 2.5: 8051 DATA TYPES AND DIRECTIVES

In this section we look at some widely used data types and directives supported by the 8051 assembler.

### 8051 data type and directives

The 8051 microcontroller has only one data type. It is 8 bits, and the size of each register is also 8 bits. It is the j of the programmer to break down data larger than 8 bits (00 to FFH, or 0 to 255 in decimal) to be processed by the C For examples of how to process data larger than 8 bits, see Chapter 6. The data types used by the 8051 can be possi or negative. A discussion of signed numbers is given in Chapter 6.

### DB (define byte)

The DB directive is the most widely used data directive in the assembler. It is used to define the 8-bit data. Wh DB is used to define data, the numbers can be in decimal, binary, hex, or ASCII formats. For decimal, the "D" after decimal number is optional, but using "B" (binary) and "H" (hexadecimal) for the others is required. Regardless which is used, the assembler will convert the numbers into hex. To indicate ASCII, simply place the characters in quo tion marks ('like this'). The assembler will assign the ASCII code for the numbers or characters automatically. The directive is the only directive that can be used to define ASCII strings larger than two characters; therefore, it should used for all ASCII data definitions. Following are some DB examples:

```
        ORG   500H
DATA1:  DB    28              ;DECIMAL(1C in hex)
DATA2:  DB    00110101B       ;BINARY (35 in hex)
DATA3:  DB    39H             ;HEX
        ORG   510H
DATA4:  DB    "2591"          ;ASCII NUMBERS
        ORG   518H
DATA6:  DB    "My name is Joe"  ;ASCII CHARACTERS
```

Either single or double quotes can be used around ASCII strings. This can be useful for strings, which contain a single quote such as "O'Leary". DB is also used to allocate memory in byte-sized chunks.

### Assembler directives

The following are some more widely used directives of the 8051.

#### ORG (origin)

The ORG directive is used to indicate the beginning of the address. The number that comes after ORG can be either in hex or in decimal. If the number is not followed by H, it is decimal and the assembler will convert it to hex. Some assemblers use ".ORG" (notice the dot) instead of "ORG" for the origin directive. Check your assembler.

#### EQU (equate)

This is used to define a constant without occupying a memory location. The EQU directive does not set aside storage for a data item but associates a constant value with a data label so that when the label appears in the program, its constant value will be substituted for the label. The following uses EQU for the counter constant and then the constant is used to load the R3 register.

```
COUNT   EQU  25
...     ....
MOV     R3,#COUNT
```

When executing the instruction "MOV R3,#COUNT", the register R3 will be loaded with the value 25 (notice the # sign). What is the advantage of using EQU? Assume that there is a constant (a fixed value) used in many different places in the program, and the programmer wants to change its value throughout. By the use of EQU, the programmer can change it once and the assembler will change all of its occurrences, rather than search the entire program trying to find every occurrence.

#### END directive

Another important pseudocode is the END directive. This indicates to the assembler the end of the source (asm) file. The END directive is the last line of an 8051 program, meaning that in the source code anything after the END directive is ignored by the assembler. Some assemblers use ".END" (notice the dot) instead of "END".

### Rules for labels in Assembly language

By choosing label names that are meaningful, a programmer can make a program much easier to read and maintain. There are several rules that names must follow. First, each label name must be unique. The names used for labels in Assembly language programming consist of alphabetic letters in both uppercase and lowercase, the digits 0 through 9, and the special characters question mark (?), period (.), at (@), underline (_), and dollar sign ($). The first character of the label must be an alphabetic character. In other words it cannot be a number. Every assembler has some reserved words that must not be used as labels in the program. Foremost among the reserved words are the mnemonics for the instructions. For example, "MOV" and "ADD" are reserved since they are instruction mnemonics. In addition to the mnemonics there are some other reserved words. Check your assembler for the list of reserved words.

### Review Questions

1. The _____ directive is always used for ASCII strings.
2. How many bytes are used by the following?
   `DATA_1: DB "AMERICA"`
3. What is the advantage in using the EQU directive to define a constant value?

Today, one can use many different programming languages, such as BASIC, Pascal, C, C++, Java, and numerous others. These languages are called *high-level languages* because the programmer does not have to be concerned with the internal details of the CPU. Whereas an *assembler* is used to translate an Assembly language program into machine code (sometimes also called *object code* or opcode for operation code), high-level languages are translated into machine code by a program called a *compiler*. For instance, to write a program in C, one must use a C compiler to translate the program into machine language. Now we look at 8051 Assembly language format and use an 8051 assembler to create a ready-to-run program.

## Structure of Assembly language

An Assembly language program consists of, among other things, a series of lines of Assembly language instructions. An Assembly language instruction consists of a mnemonic, optionally followed by one or two operands. The operands are the data items being manipulated, and the mnemonics are the commands to the CPU, telling it what to do with those items.

A given Assembly language program (see Program 2-1) is a series of statements, or lines, which are either Assembly language instructions such as ADD and MOV, or statements called directives. While instructions tell the CPU what to do, directives (also called pseudo-instructions) give directions to the assembler. For example, in the above program while the MOV and ADD instructions are commands to the CPU, ORG and END are directives to the assembler. ORG tells the assembler to place the opcode at memory location 0 while END indicates to the assembler the end of the source code. In other words, one is for the start of the program and the other one for the end of the program.

An Assembly language instruction consists of four fields:

```
[label:] mnemonic [operands] [;comment]
```

Brackets indicate that a field is optional, and not all lines have them. Brackets should not be typed in. Regarding the above format, the following points should be noted.

1. The label field allows the program to refer to a line of code by name. The label field cannot exceed a certain number of characters. Check your assembler for the rule.
2. The Assembly language mnemonic (instruction) and operand(s) fields together perform the real work of the program and accomplish the tasks for which the program was written. In Assembly language statements such as

```
ADD A,B
MOV A,#67
```

```
        ORG 0H          ;start (origin) at location 0
        MOV R5,#25H     ;load 25H into R5
        MOV R7,#34H     ;load 34H into R7
        MOV A,#0        ;load 0 into A
        ADD A,R5        ;add contents of R5 to A
                        ;now A = A + R5
        ADD A,R7        ;add contents of R7 to A
                        ;now A = A + R7
        ADD A,#12H      ;add to A value 12H
                        ;now A = A + 12H
HERE:   SJMP HERE       ;stay in this loop
        END             ;end of asm source file
```

**Program 2-1: Sample of an Assembly Language Program**

## Review Questions

1. The flag register in the 8051 is called _____.
2. What is the size of the flag register in the 8051?
3. Which bits of the PSW register are user-definable?
4. Find the CY and AC flag bits for the following code.

```
MOV A, #0FFH
ADD A, #01
```

5. Find the CY and AC flag bits for the following code.

```
MOV A, #0C2H
ADD A, #3DH
```

## SECTION 2.7: 8051 REGISTER BANKS AND STACK

The 8051 microcontroller has a total of 128 bytes of RAM. In this section we discuss the allocation of these 128 bytes of RAM and examine their usage as registers and stack.

### RAM memory space allocation in the 8051

There are 128 bytes of RAM in the 8051 (some members, notably the 8052, have 256 bytes of RAM). The 128 bytes of RAM inside the 8051 are assigned addresses 00 to 7FH. As we will see in Chapter 5, they can be accessed directly as memory locations. These 128 bytes are divided into three different groups as follows.

1. A total of 32 bytes from locations 00 to 1F hex are set aside for register banks and the stack.
2. A total of 16 bytes from locations 20H to 2FH are set aside for bit-addressable read/write memory. A detailed discussion of bit-addressable memory and instructions is given in Chapter 8.
3. A total of 80 bytes from locations 30H to 7FH are used for read and write storage, or what is normally called a *scratch pad*. These 80 locations of RAM are widely used for the purpose of storing data and parameters by 8051 programmers. We will use them in future chapters to store data brought into the CPU via I/O ports.

### Register banks in the 8051

As mentioned earlier, a total of 32 bytes of RAM are set aside for the register banks and stack. These 32 bytes are divided into 4 banks of registers in which each bank has 8 registers, R0 - R7. RAM locations from 0 to 7 are set aside for bank 0 of R0 - R7 where R0 is RAM location 0, R1 is RAM location 1, R2 is location 2, and so on, until memory location 7, which belongs to R7 of bank 0. The second bank of registers R0 - R7 starts at RAM location 08 and goes to location 0FH. The third bank of R0 - R7 starts at memory location 10H and goes to location 17H. Finally, RAM locations 18H to 1FH are set aside for the fourth bank of R0 - R7. Figure 2-6 shows how the 32 bytes are allocated into 4 banks.

As we can see from Figure 2-5, bank 1 uses the same RAM space as the stack. This is a major problem in programming the 8051. We must either not use register bank 1, or allocate another area of RAM for the stack. This will be discussed in Example 2-5.

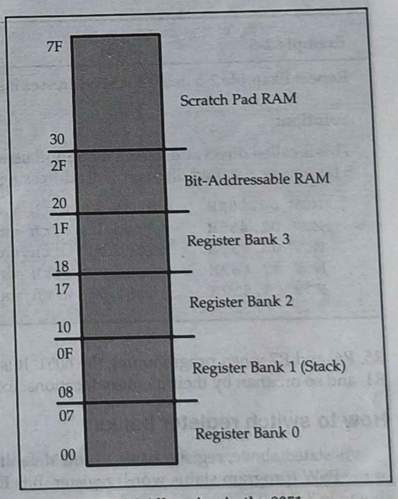



**Figure 2-5. RAM Allocation in the 8051**

### Default register bank

If RAM locations 00 - 1F are set aside for the four register banks, which register bank of R0 - R7 do we have access to when the 8051 is powered up? The answer is register bank 0; that is, RAM locations 0, 1, 2, 3, 4, 5, 6, and 7 are accessed with the names R0, R1, R2, R3, R4,

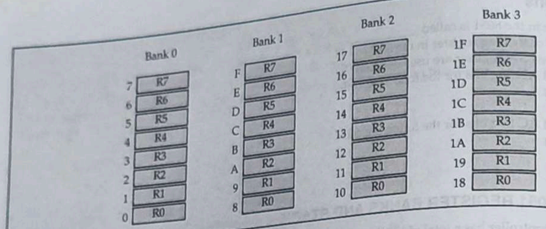



**Figure 2-6. 8051 Register Banks and their RAM Addresses**

**Example 2-5**

State the contents of RAM locations after the following program:

```
MOV R0,#99H     ;load R0 with value 99H
MOV R1,#85H     ;load R1 with value 85H
MOV R2,#3FH     ;load R2 with value 3FH
MOV R7,#63H     ;load R7 with value 63H
MOV R5,#12H     ;load R5 with value 12H
```

**Solution:**

After the execution of the above program we have the following:
RAM location 0 has value 99H RAM location 1 has value 85H
RAM location 2 has value 3FH RAM location 7 has value 63H
RAM location 5 has value 12H

**Example 2-6**

Repeat Example 2-5 using RAM addresses instead of register names.

**Solution:**

This is called direct addressing mode and uses the RAM address location for the destination address. See Chapter 5 for a more detailed discussion of addressing modes.

```
MOV 00,#99H     ;load R0 with value 99H
MOV 01,#85H     ;load R1 with value 85H
MOV 02,#3FH     ;load R2 with value 3FH
MOV 07,#63H     ;load R7 with value 63H
MOV 05,#12H     ;load R5 with value 12H
```

*R5, R6, and R7* when programming the 8051. It is much easier to refer to these RAM locations with names such as R1, and so on, than by their memory locations. Example 2-6 clarifies this concept.

### How to switch register banks

As stated above, register bank 0 is the default when the 8051 is powered up. We can switch to other banks by u of the PSW (program status word) register. Bits D4 and D3 of the PSW are used to select the desired register bank shown in Table 2-2.



The D3 and D4 bits of register PSW are often referred to as PSW.4 and PSW.3 since they can be accessed by the bit-addressable instructions SETB and CLR. For example, "SETB PSW.3" will make PSW.3 = 1 and select bank register 1. See Example 2-7.

**Table 2-2: PSW Bits Bank Selection**

| | RS1 (PSW.4) | RS0 (PSW.3) |
|---|---|---|
| Bank 0 | 0 | 0 |
| Bank 1 | 0 | 1 |
| Bank 2 | 1 | 0 |

### Stack in the 8051

The stack is a section of RAM used by the CPU to store information temporarily. This information could be data or an address. The CPU needs this storage area since there are only a limited number of registers.

### How stacks are accessed in the 8051

If the stack is a section of RAM, there must be registers inside the CPU to point to it. The register used to access the stack is called the SP (stack pointer) register. The stack pointer in the 8051 is only 8 bits wide, which means that it can take values of 00 to FFH. When the 8051 is powered up, the SP register contains value 07. This means that RAM location 08 is the first location used for the stack by the 8051. The storing of a CPU register in the stack is called a PUSH, and pulling the contents off the stack back into a CPU register is called a POP. In other words, a register is pushed onto the stack to save it and popped off the stack to retrieve it. The job of the SP is very critical when push and pop actions are performed. To see how the stack works, let's look at the PUSH and POP instructions.

### Pushing onto the stack

In the 8051 the stack pointer (SP) points to the last used location of the stack. As we push data onto the stack, the stack pointer (SP) is incremented by one. Notice that this is different from many microprocessors, notably x86 processors in which the SP is decremented when data is pushed onto the stack. Examining Example 2-8, we see that as each PUSH is executed, the contents of the register are saved on the stack and SP is incremented by 1. Notice that for every byte of data saved on the stack, SP is incremented only once. Notice also that to push the registers onto the stack we must use their RAM addresses. For example, the instruction "PUSH 1" pushes register R1 onto the stack.

**Example 2-7**

Write instructions to use the registers of bank 3, and load the same value 05H in the registers R0 to R3.

**Solution:**

```
SETB PSW.4      ;make RS1=1 for selecting bank 3
SETB PSW.3      ;make RS0=1 for selecting bank 3
MOV R0,#05H     ;load R0 with value 05H
MOV R1,#05H     ;load R1 with value 05H
MOV R2,#05H     ;load R2 with value 05H
MOV R3,#05H     ;load R3 with value 05H
```

After execution, the four registers R0 to R4 of bank 3 (RAM locations 18H to 1BH) will contain the value 05H. The program can be rewritten as

```
SETB PSW.4      ;make RS1=1 for selecting bank 3
SETB PSW.3      ;make RS0=1 for selecting bank 3
MOV A,#05H      ;load A with 05H
MOV R0,A        ;move the contents of A to R0
MOV R1,A        ;move the contents of A to R1
MOV R2,A        ;move the contents of A to R2
MOV R3,A        ;move the contents of A to R3
```

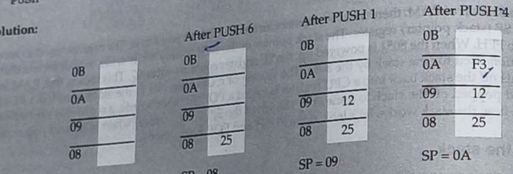
We will see later that it will be more economical to use the above set of instructions, as a register to register transfer is a shorter instruction and thus saves program memory space.

**Example 2-8**

Show the stack and stack pointer for the following. Assume the default stack area and register 0 is selected.

```
MOV   R6, #25H
MOV   R1, #12H
MOV   R4, #0F3H
PUSH  6
PUSH  1
PUSH  4
```

**Solution:**




### Popping from the stack

Popping the contents of the stack back into a given register is the opposite process of pushing. With every POP, top byte of the stack is copied to the register specified by the instruction and the stack pointer is decremented on Example 2-9 demonstrates the POP instruction.

### The upper limit of the stack

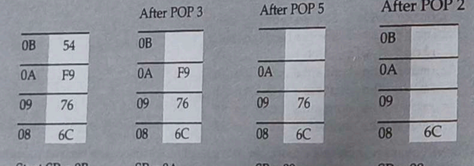
As mentioned earlier, locations 08 to 1F in the 8051 RAM can be used for the stack. This is because locations 20 - 2 of RAM are reserved for bit-addressable memory and must not be used by the stack. If in a given program we ne

**Example 2-9**

Examining the stack, show the contents of the registers and SP after execution of the following instructions. Al values are in hex.

```
POP 3    ;POP stack into R3
POP 5    ;POP stack into R5
POP 2    ;POP stack into R2
```

**Solution:**






more than 24 bytes (08 to 1FH = 24 bytes) of stack, we can change the SP to point to RAM locations 30 - 7FH. This is done with the instruction "MOV SP, #xx".

### CALL instruction and the stack

In addition to using the stack to save registers, the CPU also uses the stack to save the address of the instruction just below the CALL instruction. This is how the CPU knows where to resume when it returns from the called subroutine. More information on this will be given in Chapter 3 when we discuss the CALL instruction.

### Stack and bank 1 conflict

Recall from our earlier discussion that the stack pointer register points to the current RAM location available for the stack. As data is pushed onto the stack, SP is incremented. Conversely, it is decremented as data is popped off the stack into the registers. The reason that the SP is incremented after the push is to make sure that the stack is growing toward RAM location 7FH, from lower addresses to upper addresses. If the stack pointer were decremented after push instructions, we would be using RAM locations 7, 6, 5, etc., which belong to R7 to R0 of bank 0, the default register bank. This incrementing of the stack pointer for push instructions also ensures that the stack will not reach location 0 at the bottom of RAM, and consequently run out of space for the stack. However, there is a problem with the default setting of the stack. Since SP = 07 when the 8051 is powered up, the first location of the stack is RAM location 08, which also belongs to register R0 of register bank 1. In other words, register bank 1 and the stack are using the same memory space. If in a given program we need to use register banks 1 and 2, we can reallocate another section of RAM to the stack. For example, we can allocate RAM locations 60H and higher to the stack as shown in Example 2-10.

**Example 2-10**

Write PUSH instructions to push the contents of the registers on stack after the execution of the following set of instructions.

```
MOV SP,#4FH
SETB PSW.3
MOV R0,#25H
MOV R1,#0CH
MOV R2,#05H
MOV A, #0CEH
```

**Solution:**

The first instruction defines the stack to be from 50H onwards. The second instruction defines the use of register bank 1. Since the register bank 1 has been selected, the addresses of registers R0, R1, and R2 are 8, 9, and 0AH. The address of the A register is 0E0H. Hence the PUSH instructions are

```
PUSH 8
PUSH 9
PUSH 0AH
PUSH 0E0H
```

After the four PUSH instructions, the content of the RAM selected as the stack locations will be

| Address | Data |
|---|---|
| 50 | 25H |
| 51 | 0CH |
| 52 | 05H |
| 53 | CEH |

This chapter describes the process of physically connecting and testing 8051-based systems. In the first section we describe the function of the pins of 8051 chip. The second section shows the hardware connection for an 8051 Trainer using the DS89C4x0 (DS89C420/30/40/50) chip. It also shows how to download programs into a DS89C4x0-based system using PC HyperTerminal. In Section 8.3 we explain the characteristics of the Intel hex file.

## SECTION 8.1: PIN DESCRIPTION OF THE 8051

Although 8051 family members (e.g., 8751, 89C51, 89C52, DS89C4x0) come in different packages, such as DIP (dual in-line package), QFP (quad flat package), and LLC (leadless chip carrier), they all have 40 pins that are dedicated to various functions such as I/O, $\overline{RD}$, $\overline{WR}$, address, data, and interrupts. It must be noted that some companies provide a 20-pin version of the 8051 with a reduced number of I/O ports for less demanding applications. However, since the vast majority of developers use the 40-pin chip, we will concentrate on that. Figure 8-1 shows the pins for the 8051/52. For the 8052 chip some of the pins have extra functions and they will be discussed as we study them.

Examining Figure 8-1, note that of the 40 pins, a total of 32 pins are set aside for the four ports P0, P1, P2, and P3, where each port takes 8 pins. The rest of the pins are designated as $V_{cc}$, GND, XTAL1, XTAL2, RST, $\overline{EA}$, $\overline{PSEN}$, and ALE. Of these pins, six ($V_{cc}$, GND, XTAL1, XTAL2, RST, and $\overline{EA}$) are used by all members of the 8051 and 8031 families. In other words, they must be connected in order for the system to work, regardless of whether the microcontroller is of the 8051 or 8031 family. The other two pins, $\overline{PSEN}$ and ALE, are used mainly in 8031-based systems. We first describe the function of each pin. Ports are discussed separately.

### $V_{cc}$

Pin 40 provides supply voltage to the chip. The voltage source is +5V.

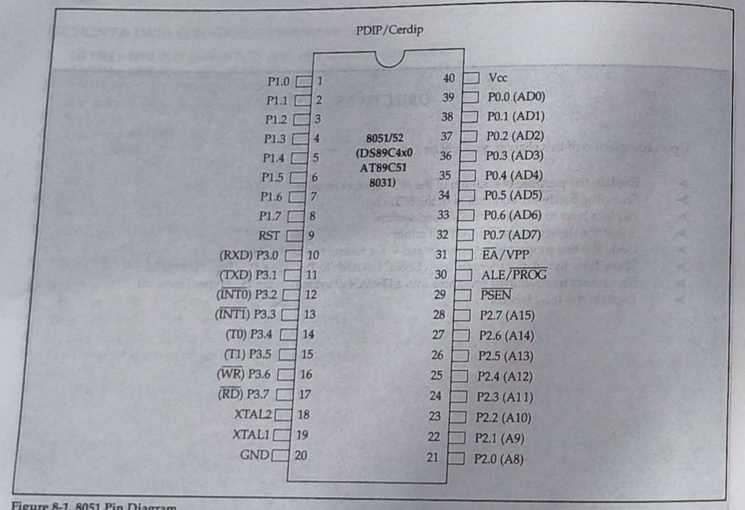



Figure 8-1. 8051 Pin Diagram



### GND

Pin 20 is the ground.

### XTAL1 and XTAL2

The 8051 has an on-chip oscillator but requires an external clock to run it. Most often a quartz crystal oscillator is connected to inputs XTAL1 (pin 19) and XTAL2 (pin 18). The quartz crystal oscillator connected to XTAL1 and XTAL2 also needs two capacitors of 30 pF value. One side of each capacitor is connected to the ground as shown in Figure 8-2 (a).

It must be noted that there are various speeds of the 8051 family. Speed refers to the maximum oscillator frequency connected to XTAL. For example, a 12-MHz chip must be connected to a crystal with 12 MHz frequency or less. Likewise, a 20-MHz microcontroller requires a crystal frequency of no more than 20 MHz. When the 8051 is connected to a crystal oscillator and is powered up, we can observe the frequency on the XTAL2 pin using the oscilloscope.

If you decide to use a frequency source other than a crystal oscillator, such as a TTL oscillator, it will be connected to XTAL1; XTAL2 is left unconnected, as shown in Figure 8-2 (b).

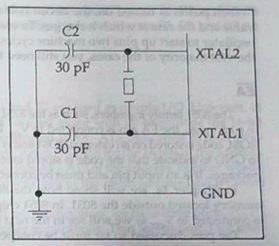



Figure 8-2 (a). XTAL Connection to 8051

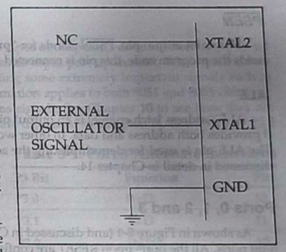



Figure 8-2 (b). XTAL Connection to an External Clock Source

### RST

Pin 9 is the RESET pin. It is an input and is active high (normally low). Upon applying a high pulse to this pin, the microcontroller will reset and terminate all activities. This is often referred to as a *power-on reset*. Activating a power-on reset will cause all values in the registers to be lost. It will set program counter to all 0s.

Figures 8-3 (a) and (b) show two ways of connecting the RST pin to the power-on reset circuitry. Figure 8-3 (b) uses a momentary switch for reset circuitry.

In order for the RESET input to be effective, it must have a minimum duration of two machine cycles. In other words, the high pulse must be high for a minimum of two machine cycles before it is allowed to go low. Here is what the Intel manual says about the Reset circuitry:

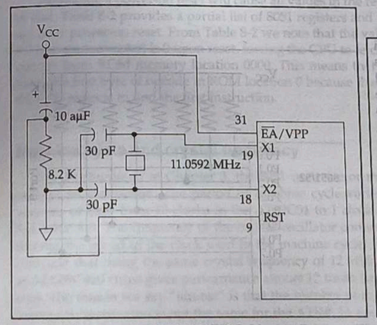



Figure 8-3 (a). Power-On RESET Circuit

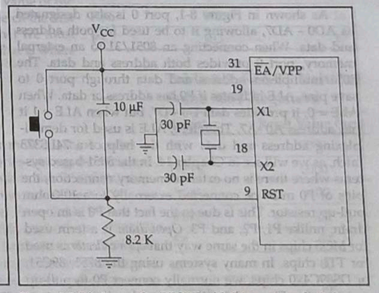



Figure 8-3 (b). Power-On RESET with Momentary Switch

"When power is turned on, the circuit holds the RST pin high for an amount of time that depends on the capacitor value and the rate at which it charges. To ensure a valid reset the RST pin must be held high long enough to allow the oscillator to start up plus two machine cycles." Although, an 8.2K-ohm resistor and a 10-µF capacitor will take care of the vast majority of the cases, you still need to check the data sheet for the 8051 you are using.

### EA

The 8051 family members, such as the 8751/52, 89C51/52, or DS89C4x0, all come with on-chip ROM to store programs. In such cases, the EA pin is connected to $V_{CC}$. For family members such as the 8031 and 8032 in which there is no on-chip ROM, code is stored on an external ROM and is fetched by the 8031/32. Therefore, for the 8031 the EA pin must be connected to GND to indicate that the code is stored externally. EA, which stands for "external access," is pin number 31 in the DIP packages. It is an input pin and must be connected to either $V_{CC}$ or GND. In other words, it cannot be left unconnected.

In Chapter 14, we will show how the 8031 uses this pin along with PSEN to access programs stored in ROM memory located outside the 8031. In 8051 chips with on-chip ROM, such as the 8751/52, 89C51/52, or DS89C4 x0, EA is connected to $V_{CC}$, as we will see in the next section.

The pins discussed so far must be connected no matter which family member is used. The next two pins are used mainly in 8031-based systems and are discussed in more detail in Chapter 14. The following is a brief description of each.

### PSEN

This is an output pin. PSEN stands for "program store enable." In an 8031-based system in which an external ROM holds the program code, this pin is connected to the OE pin of the ROM. See Chapter 14 to see how this is used.

### ALE

ALE (address latch enable) is an output pin and is active high. When connecting an 8031 to external memory, port 0 provides both address and data. In other words, the 8031 multiplexes address and data through port 0 to save pins. The ALE pin is used for demultiplexing the address and data by connecting to the G pin of the 74LS373 chip. This is discussed in detail in Chapter 14.

## Ports 0, 1, 2 and 3

As shown in Figure 8-1 (and discussed in Chapter 4), the four ports P0, P1, P2, and P3 each use 8 pins, making them 8-bit ports. All the ports upon RESET are configured as input, since P0 - P3 have value FFH on them. The following is a summary of features of P0 - P3 based on the materials in Chapter 4.

### P0

As shown in Figure 8-1, port 0 is also designated as AD0 - AD7, allowing it to be used for both address and data. When connecting an 8051/31 to an external memory, port 0 provides both address and data. The 8051 multiplexes address and data through port 0 to save pins. ALE indicates if P0 has address or data. When ALE = 0, it provides data D0 - D7, but when ALE = 1 it has address A0 -A7. Therefore, ALE is used for demultiplexing address and data with the help of a 74LS373 latch, as we will see in Chapter 14. In the 8051-based systems where there is no external memory connection, the pins of P0 must be connected externally to a 10K-ohm pull-up resistor. This is due to the fact that P0 is an open drain, unlike P1, P2, and P3. *Open drain* is a term used for MOS chips in the same way that *open collector* is used for TTL chips. In many systems using the 8751, 89C51, or DS89C4x0 chips, we normally connect P0 to pull-up resistors. See Figure 8-4. With external pull-up resistors

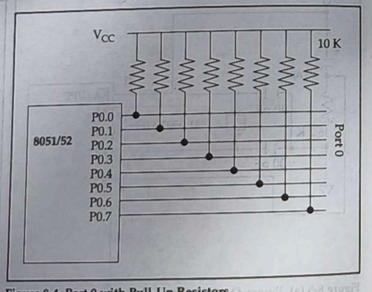



**Figure 8-4. Port 0 with Pull-Up Resistors**





connected to P0, it can be used as a simple I/O port, just like P1 and P2. In contrast to port 0, ports P1, P2, and P3 do not need any pull-up resistors since they already have pull-up resistors internally. Upon reset, ports P1, P2, and P3 are configured as input ports.

### P1 and P2

In 8051-based systems with no external memory connection, both P1 and P2 are used as simple I/O. However, in 8031/51-based systems with external memory connections, port 2 must be used along with P0 to provide the 16-bit address for the external memory. As shown in Figure 8-1, port 2 is also designated as A8 - A15, indicating its dual function. Since an 8031/51 is capable of accessing 64K bytes of external memory, it needs a path for the 16 bits of the address. While P0 provides the lower 8 bits via A0 - A7, it is the job of P2 to provide bits A8 - A15 of the address. In other words, when the 8031/51 is connected to external memory, P2 is used for the upper 8 bits of the 16-bit address, and it cannot be used for I/O. This is discussed in detail in Chapter 14.

From the discussion so far, we conclude that in systems based on 8051 microcontrollers, we have three ports, P0, P1, and P2, for I/O operations. This should be enough for most microcontroller applications. That leaves port 3 for interrupts as well as other signals, as we will see next.

### Port 3

Port 3 occupies a total of 8 pins, pins 10 through 17. It can be used as input or output. P3 does not need any pull-up resistors, the same as P1 and P2 did not. Although port 3 is configured as an input port upon reset, this is not the way it is most commonly used. Port 3 has the additional function of providing some extremely important signals such as interrupts. Table 8-1 provides these alternate functions of P3. This information applies to both 8051 and 8031 chips.

P3.0 and P3.1 are used for the R×D and T×D serial communications signals. See Chapter 10 to see how they are connected. Bits P3.2 and P3.3 are set aside for external interrupts, and are discussed in Chapter 11. Bits P3.4 and P3.5 are used for Timers 0 and 1, and are discussed in Chapter 9. Finally, P3.6 and P3.7 are used to provide the WR and RD signals of external memory connections. Chapter 14 discusses how they are used in 8031-based systems. In systems based on the 8051, pins 3.6 and 3.7 are used for I/O while the rest of the pins in port 3 are normally used in the alternate function role.

**Table 8-1: Port 3 Alternate Functions**

| P3 Bit | Function | Pin |
|---|---|---|
| P3.0 | R×D | 10 |
| P3.1 | T×D | 11 |
| P3.2 | $\overline{INT0}$ | 12 |
| P3.3 | $\overline{INT1}$ | 13 |
| P3.4 | T0 | 14 |
| P3.5 | T1 | 15 |
| P3.6 | WR | 16 |
| P3.7 | $\overline{RD}$ | 17 |

### Program counter value upon reset

Activating a power-on reset will cause all values in the registers to be lost. Table 8-2 provides a partial list of 8051 registers and their values after power-on reset. From Table 8-2 we note that the value of the PC (program counter) is 0 upon reset, forcing the CPU to fetch the first opcode from ROM memory location 0000. This means that we must place the first byte of opcode in ROM location 0 because that is where the CPU expects to find the first instruction.

**Table 8-2: RESET Value of Some 8051 Registers**

| Register | Reset Value (hex) |
|---|---|
| PC | 0000 |
| DPTR | 0000 |
| ACC | 00 |
| PSW | 00 |
| SP | 07 |
| B | 00 |
| P0-P3 | FF |

### Machine cycle and crystal frequency

As we discussed in Chapter 3, the 8051 uses one or more machine cycles to execute an instruction. The period of machine cycle varies among the different versions of 8051 from 12 clocks in the AT89C51 to 1 clock in the DS89C4x0 chip. See Table 8-3. The frequency of the crystal oscillator connected to the X - X2 pins dictates the speed of the clock used in the machine cycle. From Table 8-3, we can conclude that using the same crystal frequency of 12 MHz for both the AT89C51 and DS89C4x0 chips gives performance almost 12 times better from the DS89C4x0 chip. The reason we say "almost" is that the number of machine cycles it takes to execute an instruction is not the same for the AT89C51 and DS89C4x0 chips as we discussed in Chapter 3.